

CSA 1709 - Artificial Intelligence - Slot: B

Lab Experiments

Experiment 17: Prolog Program to Sum the Integers from 1 to N

Aim: To write a Prolog program to find the sum of integers from 1 to N.

Algorithm:

1. Start the program.
2. Define a predicate `sum(N, S)` to calculate the sum of integers from 1 to N.
3. If $N = 0$, set the sum as 0.
4. Otherwise, recursively calculate the sum up to $N-1$.
5. Add N to the previous sum.
6. Display the result.
7. Stop.

Program:

```
sum(0, 0).  
sum(N, S) :-  
    N > 0,  
    N1 is N - 1,  
    sum(N1, S1),  
    S is N + S1.
```

Query:

```
?- sum(10, S).
```

Output:

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/sum n numbers.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/sum n numbers.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/sum n numbers.pl compiled, 5 lines read - 854 bytes written, 6 ms

(31 ms) yes
| ?- sum(10,Sum).

Sum = 55 ?
```

Result: The sum of integers from 1 to 10 is 55.

Experiment 18. Prolog Program for a Database with Name and DOB

Aim: To write a Prolog program to create a database containing Name and Date of Birth (DOB) and retrieve the DOB of a person.

Algorithm:

1. Start the program.
2. Define facts containing the person's name and date of birth.
3. Define a predicate `dob(Name, Date)` to retrieve the DOB.
4. Query the database using a person's name.
5. Display the corresponding DOB.
6. Stop.

Program:

```
dob(john, date(1990, 5, 1)).
dob(jane, date(1985, 12, 10)).
dob(bob, date(1978, 2, 28)).
dob(sue, date(1995, 8, 15)).
dob(tom, date(2000, 4, 22)).
lookup(Name, DOB) :-
    dob(Name, DOB).
```

Query:

```
?- lookup(john, DOB).
```

Output:

```
(78 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/DOBs.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/DOBs.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/DOBs.pl compiled, 6 lines read - 1193 bytes written, 7 ms

(15 ms) yes
| ?- lookup(john, DOB).

DOB = date(1990,5,1)
```

Result: The DOB of the specified person is successfully retrieved from the database.

Experiment 19. Prolog Program for Student-Teacher-Sub-Code

Aim: To write a Prolog program to represent the relationship between students, teachers, and subjects/codes and retrieve the corresponding information.

Algorithm:

1. Start the program.
2. Define facts for students and their subjects.
3. Define facts for teachers and the subjects they handle.
4. Define subject codes.
5. Create a predicate to relate student, teacher, and subject code.
6. Query the database.
7. Display the result.
8. Stop.

Program:

```
teaches(john, math). teaches(jane, english).
teaches(bob, science). teaches(sue, history).
teaches(tom, art).
takes(alice, math).
takes(alice, science).
takes(bob, english).
takes(bob, science).
takes(carol, history).
takes(carol, art).
takes(dave, math).
takes(dave, english).
takes(dave, art).
teaching_subjects(Teacher, Subject) :-
    teaches(Teacher, Subject).
taking_students(Subject, Student) :-
    takes(Student, Subject).
```

Query:

```
?- teaching_subjects(john, Subject).
?- taking_students(science, Student).
?- taking_students(math, Student), taking_students(art, Student).
```

Output

```
yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Stud-Teach.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Stud-Teach.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Stud-Teach.pl compiled, 15 lines read - 1912 bytes written, 11 ms

(15 ms) yes
| ?- teaching_subjects(john, Subject).

Subject = math

yes
| ?- taking_students(science, Student).

Student = alice ? ;

Student = bob ? .
Action (; for next solution, a for all solutions, RET to stop) ?

(47 ms) yes
| ?- taking_students(math, Student), taking_students(art, Student).

Student = dave ? ;

(31 ms) no
```

Result: The student, teacher, and corresponding subject code are successfully obtained using Prolog relationships.

Experiment 20. Prolog Program for Planets Database

Aim: To write a Prolog program to create a database of planets and retrieve information about them.

Algorithm:

1. Start the program.
2. Define facts containing the names of planets.
3. Store information such as order, type, and number of moons.
4. Define predicates to retrieve planet information.
5. Query the database.
6. Display the result.
7. Stop.

Program:

```
planet(mercury, 1, terrestrial, 0).  
planet(venus, 2, terrestrial, 0).  
planet(earth, 3, terrestrial, 1).  
planet(mars, 4, terrestrial, 2).  
planet(jupiter, 5, gas_giant, 95).  
planet(saturn, 6, gas_giant, 146).  
planet(uranus, 7, ice_giant, 28).  
planet(neptune, 8, ice_giant, 16).  
planet_info(Name, Order, Type, Moons) :-  
    planet(Name, Order, Type, Moons).
```

Query:

```
?- planet_info(earth, Order, Type, Moons).  
?- planet_info(jupiter, Order, Type, Moons).
```

Output:

```
GNU Prolog console
File Edit Terminal Prolog Help
GNU Prolog 1.5.0 (64 bits)
Compiled Jul  8 2021, 12:33:56 with cl
Copyright (C) 1999-2021 Daniel Diaz

| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/PlanetData.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/PlanetData.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/PlanetData.pl compiled, 9 lines read - 1587 bytes written, 7 ms

yes
| ?- planet_info(earth, Order, Type, Moons).

Moons = 1
Order = 3
Type = terrestrial

yes
| ?- planet_info(jupiter, Order, Type, Moons).

Moons = 95
Order = 5
Type = gas_giant
```

Result: The planet database is successfully created and the required information is retrieved using Prolog.

Experiment 21. Prolog Program to Implement Towers of Hanoi

Aim

To write a Prolog program to implement the Towers of Hanoi problem using recursion.

Algorithm

1. Start with N disks on the source peg.
2. If there is only one disk, move it directly from source to destination.
3. Otherwise:
 - Move $N-1$ disks from source to auxiliary peg.
 - Move the largest disk from source to destination.
 - Move $N-1$ disks from auxiliary peg to destination.
4. Display all the required moves.
5. Stop.

Program

hanoi(1, Source, Destination, _) :-

write('Move disk from '),

write(Source),

write(' to '),

write(Destination),

nl.

hanoi(N, Source, Destination, Auxiliary) :-

$N > 1$,

$N1$ is $N - 1$,

hanoi($N1$, Source, Auxiliary, Destination),

write('Move disk from '),

write(Source),

```
write(' to '),  
  
write(Destination),  
  
nl,  
  
hanoi(N1, Auxiliary, Destination, Source).
```

Query

```
?- hanoi(3, source, destination, auxiliary).
```

Output

```
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Tower of Hanoi.pl').  
compiling C:/Users/Vidhushini/OneDrive/Documents/Tower of Hanoi.pl for byte code...  
C:/Users/Vidhushini/OneDrive/Documents/Tower of Hanoi.pl compiled, 15 lines read - 1604 bytes written, 7 ms  
  
(16 ms) yes  
| ?- hanoi(3, source, destination, auxiliary).  
Move disk from source to destination  
Move disk from source to auxiliary  
Move disk from destination to auxiliary  
Move disk from source to destination  
Move disk from auxiliary to source  
Move disk from auxiliary to destination  
Move disk from source to destination  
  
true ? |
```

Result

Thus, the Towers of Hanoi problem was successfully implemented using Prolog.

22. Prolog Program to Check Whether a Particular Bird Can Fly or Not

Aim

To write a Prolog program to determine whether a particular bird can fly or cannot fly using facts and rules.

Algorithm

1. Start the program.
2. Define facts for birds that can fly.
3. Define facts for birds that cannot fly.
4. Define the predicate `can_fly(Bird)`.
5. Query the required bird.
6. Display whether the bird can fly or not.
7. Stop.

Program

```
can_fly(eagle).
```

```
can_fly(parrot).
```

```
can_fly(pigeon).
```

```
can_fly(crow).
```

```
can_fly(sparrow).
```

```
cannot_fly(penguin).
```

```
cannot_fly(ostrich).
```

```
cannot_fly(emu).
```

```
bird_can_fly(Bird) :-
```

```
    can_fly(Bird).
```

```
bird_cannot_fly(Bird) :-
```

```
    cannot_fly(Bird).
```

Query 1

?- bird_can_fly(eagle).

?- bird_cannot_fly(penguin).

?- bird_can_fly(penguin).

Output:

```
(172 ms) yes
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Bird.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Bird.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Bird.pl compiled, 11 lines read - 1084 bytes written, 8 ms

(31 ms) yes
| ?- bird_can_fly(eagle).

yes
| ?- bird_cannot_fly(penguin).

yes
| ?- bird_can_fly(penguin).

no
| ?-
```

Result

Thus, the Prolog program successfully determines whether a particular bird can fly or not.

Experiment 23. Prolog Program to Implement Family Tree

Aim

To write a Prolog program to represent a family tree and determine relationships such as parent, grandparent, sibling, and child.

Algorithm

1. Start the program.
2. Define facts for parent-child relationships.
3. Define the father and mother relationships.
4. Define rules for grandparent.
5. Define rules for sibling.
6. Query the required relationship.
7. Display the result.
8. Stop.

Program

```
father(ramesh, rahul).
```

```
father(ramesh, priya).
```

```
father(suresh, anjali).
```

```
mother(sita, rahul).
```

```
mother(sita, priya).
```

```
mother(geetha, anjali).
```

```
parent(X, Y) :-
```

```
    father(X, Y).
```

```
parent(X, Y) :-
```

```
    mother(X, Y).
```

```
grandparent(X, Z) :-
```

```

parent(X, Y),
parent(Y, Z).

sibling(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.

```

Query

```

?- parent(X, rahul).

?- sibling(rahul, X).

?- grandparent(X, rahul).

```

Output:

```

| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Relation.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Relation.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Relation.pl compiled, 16 lines read - 1760 bytes written, 9 ms

yes
| ?- parent(X,rahul).

X = ramesh ? ;

X = sita ? .
Action (; for next solution, a for all solutions, RET to stop) ?

(15 ms) yes
| ?- sibling(rahul,X).

X = priya ? .
Action (; for next solution, a for all solutions, RET to stop) ?

(16 ms) yes
| ?- grandparent(X,rahul).

no
| ?- |

```

Result

Thus, the family tree was successfully implemented using Prolog facts and rules.

Experiment 24. Prolog Program to Suggest Dieting System Based on Disease

Aim: To write a Prolog program to suggest a suitable dieting system based on a person's disease.

Algorithm

1. Start the program.
2. Define diseases and recommended foods.
3. Define diseases and foods to avoid.
4. Create a predicate to suggest a diet.
5. Query the disease.
6. Display the recommended foods and foods to avoid.

Program

recommended_diabetes(vegetables).

recommended_diabetes(whole_grains).

recommended_diabetes(fruits_in_moderation).

avoid_diabetes(sugary_foods).

avoid_diabetes(soft_drinks).

recommended_hypertension(fruits).

recommended_hypertension(vegetables).

recommended_hypertension(low_salt_food).

avoid_hypertension(salty_foods).

avoid_hypertension(processed_foods).

recommended_anemia(leafy_vegetables).

recommended_anemia(beetroot).

recommended_anemia(iron_rich_foods).

avoid_anemia(excess_tea).

avoid_anemia(excess_coffee).

diet(Disease, Food) :-

Disease = diabetes,

recommended_diabetes(Food).

diet(Disease, Food) :-

Disease = hypertension,

recommended_hypertension(Food).

diet(Disease, Food) :-

Disease = anemia,

recommended_anemia(Food).

Query :

?- diet(diabetes, Food).

?- diet(hypertension, Food).

Output:

```
--
| ?- consult('C:/Users/Vidhushini/OneDrive/Documents/Diet System.pl').
compiling C:/Users/Vidhushini/OneDrive/Documents/Diet System.pl for byte code...
C:/Users/Vidhushini/OneDrive/Documents/Diet System.pl compiled, 23 lines read - 2467 bytes written, 11 ms

(15 ms) yes
| ?- diet(diabetes,Food).

Food = vegetables ? ;
Food = whole_grains ? ;
Food = fruits_in_moderation ? .
Action (; for next solution, a for all solutions, RET to stop) ?

yes
| ?- diet(hypertension,Food).

Food = fruits ? ;
Food = vegetables ? ;
Food = low_salt_food ? .
Action (; for next solution, a for all solutions, RET to stop) ?
```

Result: Thus, a Prolog-based diet suggestion system was successfully implemented based on disease.